



Pontifícia Universidade Católica de Minas Gerais
Bacharelado em Engenharia de *Software* - *Campus* Coração Eucarístico
Algoritmos e Estruturas de Dados II (Laboratório)
Prof. João Pedro Oliveira Batisteli - Semestre 1/2026
Atividade: Estruturas de Dados Lineares em Sistemas de *Software* -
Valor: 3 pontos - Data: 11/05/2026

INSTRUÇÕES

- Esta é uma **atividade individual**, em laboratório, da disciplina “Algoritmos e Estruturas de Dados II”.
- Utilize seu repositório, criado no **GitHub Classroom**, para resolver as questões de programação propostas e enviar suas respostas para correção. **Nenhuma outra forma de entrega da atividade será aceita.**
- O prazo **limite para entrega** é o dia **11/05**, às **12h20**. *Commits* no repositório com data posterior a esse limite serão desconsiderados no momento da correção da atividade.
- Além do **conteúdo do seu repositório de trabalho** e dos **materiais disponíveis no Canvas citados na Tarefa 0**, **não é permitida a consulta a nenhum outro material**, porém a professora está **disponível para esclarecer dúvidas** e debater propostas de soluções vindas dos alunos.
- Sendo uma atividade individual, **espera-se dos alunos comprometimento e ética** na produção de sua solução. Soluções produzidas em conjunto, entregas contendo cópias, mesmo que parciais, em código ou estrutura, e consultas a materiais e ferramentas não permitidos são **faltas graves, resultando em nota 0 para todos os envolvidos** e posterior **notificação à Coordenação do Curso de Engenharia de Software** por desrespeito às normas acadêmicas.

Tema: Estruturas de dados lineares em sistemas de *software*

Temos um sistema de software simples que permite a venda de produtos de escritório agrupados em pedidos, com funcionalidades de consulta aos produtos cadastrados, recuperação de dados de arquivos-texto, e algumas funções gerenciais implementadas recentemente com uso de pilhas e filas. Vamos agora utilizar uma nova estrutura de dados, listas encadeadas, melhorar nossa classe “Pedido” empregando essa estrutura e incluir uma nova função gerencial no sistema usando uma lista de pedidos.

Tarefa 0: (preparação)

- **Adicione uma nova ramificação (branch)** no seu repositório GitHub Classroom correspondente a esta atividade, com o nome **atividade1105**, a partir de seu *branch* anterior. Certifique-se de que as classes **Pedido**, **ItemDePedido**, **Celula**, **Pilha** e **Fila** estejam neste novo *branch*.
- Em seguida, **faça o download do código-base** fornecido no Canvas e inclua a nova classe **Lista** em sua pasta de código.
- Estude o código da classe **Lista**. Esta classe tem as operações básicas de uma lista simplesmente encadeada e algumas outras operações úteis para o uso dessa estrutura de dados, a serem implementadas nessa atividade.

Tarefa 1: (0,5 ponto)

Implemente o método `public E buscarPor(Comparator<E> criterioDeBusca, E item)` na classe `Lista<E>`. Esse método deve ser capaz de buscar, na lista, e retornar o primeiro elemento correspondente ao item informado como parâmetro, utilizando o critério de comparação especificado. O método deve percorrer sequencialmente os elementos da lista e utilizar o **Comparator** informado como parâmetro para determinar se um elemento da lista é equivalente ao item procurado. O parâmetro `criterioDeBusca` indica o critério que deve ser empregado para comparar os elementos da lista com o item procurado. Já o parâmetro `item` corresponde ao elemento que será buscado na lista. Caso nenhum elemento correspondente ao item procurado seja encontrado, esse método deve retornar `null`.

Tarefa 2: (0,5 ponto)

Implemente o método `public double somarMultiplicacoes(Function<E, Double> extratorValor, Function<E, Integer> extratorFator)` na classe `Lista<E>`. Esse método deve ser capaz de calcular o somatório do produto entre dois valores extraídos de cada elemento armazenado na lista. Para cada elemento da lista, o método deve: extrair um valor numérico do tipo `double`; extrair um valor inteiro utilizado como fator

multiplicador; multiplicar os valores obtidos; acumular o resultado no somatório final. O resultado retornado corresponde ao somatório do produto entre os valores extraídos de cada elemento. Apresenta os parâmetros **extratorValor**, responsável por extrair o valor numérico do elemento da lista; e **extratorFator**; responsável por extrair o fator multiplicador do elemento. Esse método deve lançar a exceção **IllegalStateException** se a lista estiver vazia.

Tarefa 3: (0,5 ponto)

Implemente o método **public Lista<E> filtrar(Predicate<E> condicional)** na classe **Lista<E>**. Esse método deve retornar uma nova lista contendo apenas os elementos da lista original que satisfazem uma condição específica indicada pelo parâmetro **condicional**. Esse método deve lançar a exceção **IllegalStateException** se a lista estiver vazia.

Um **Predicate<T>** é uma interface funcional do pacote **java.util.function** que representa uma condição aplicada a um objeto do tipo **T**. Seu único método abstrato é **test(T t)**, que retorna **true** caso o objeto satisfaça a condição definida, ou **false** caso contrário. **Predicate** é frequentemente utilizado para filtrar elementos de uma coleção: passamos a condição como parâmetro e a estrutura de dados decide, para cada elemento, se ele deve ou não compor o resultado. **Exemplo de uso:**

```
// Predicate que verifica se uma String tem mais de 5 caracteres
Predicate<String> maisQueCincoCaracteres = s -> s.length() > 5;
System.out.println(maisQueCincoCaracteres.test("oi")); // false
System.out.println(maisQueCincoCaracteres.test("caderno")); // true
```

No exemplo acima, a expressão lambda **s -> s.length() > 5** define a condição: para cada String **s**, o **Predicate** retorna **true** se o comprimento for maior que 5. O mesmo princípio se aplica ao método **filtrar** da classe **Lista<E>**, que recebe um **Predicate<E>** e retorna uma nova lista apenas com os elementos que satisfazem a condição. No código-base fornecido você encontrará um exemplo concreto de **Predicate** sendo passado para o método **filtrar**, utilize-o como referência para a sua implementação.

Tarefa 4: (1 ponto)

Refatore o código da classe **Pedido** de forma que um pedido, agora, tenha uma **lista** de itens de pedido. Realize todas as alterações necessárias nos diversos métodos dessa classe para contemplar essa modificação. Lembre-se de empregar os métodos da classe **Lista<E>** que foram implementados nas tarefas anteriores ou que já estavam implementados no código-base fornecido.

Tarefa 5: (0,5 ponto)

Com a refatoração da classe **Pedido**, é possível que alguns métodos implementados anteriormente na classe **App** parem de funcionar. Para essa atividade, não é necessário refatorar os métodos da classe **App**. Os métodos dessa classe que pararem de funcionar após a refatoração da classe **Pedido** podem ser comentados ou removidos. Altere o código da classe **App** desenvolvido na atividade anterior permitindo agora que os pedidos sejam armazenados em uma lista.

Acrescente ao **menu** do sistema de comércio a seguinte nova opção: **Exibir os pedidos que apresentam um determinado produto**. Implemente o método correspondente a essa nova funcionalidade do sistema de comércio, a saber:

- **public static void filtrarPorProduto():** filtra e exibe os pedidos que apresentam um produto específico, cuja descrição foi informada pelo usuário. O método deve **obrigatoriamente** utilizar a função **filtrar**, da classe **Lista<E>**, para determinar se o pedido deve ou não compor a resposta final; e o método **buscarPor**, também da classe **Lista<E>**, para verificar se existe um item de pedido cujo produto possui descrição equivalente à informada pelo usuário.

Instruções e observações:

- O projeto deve estar hospedado na tarefa correspondente do GitHub Classroom. Endereço para aceitar a tarefa: <https://classroom.github.com/a/hF6FnYcs>
- As atividades pontuadas da disciplina podem depender direta ou indiretamente dos códigos desenvolvidos nas aulas. Portanto, é essencial o comprometimento no acompanhamento das atividades semanais.
- Para a correção das atividades pontuadas, serão considerados todos os *commits/pushes* realizados ao

longo das semanas, não somente o último com a resposta final do exercício.